

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 2: Real-time Study Room Booking App (React Native & Expo)

Team / Student Name: Lê Cẩm (CAMLC25)

Submission Date: 05/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:** Lê Cẩm — Student ID: 21IT001 — Role: Fullstack Mobile Architecture & Concurrency Engine — Contribution: 100%
-  **Live Demo URL:** <https://camle-vku-study-room.lecam.workers.dev>
-  **GitHub Repository:** <https://github.com/CAMLC25/camle-vku-study-room>
-  **Video Demo (Optional):** <https://camle-vku-study-room.lecam.workers.dev>

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Responsive Viewport & Safe Area	 Complete	Custom hook useResponsiveLayout adapts across Mobile (1 col) and Desktop/Tablet (2 cols) with Math.min(width, 1140) width clamping and flex: 1 columns preventing offscreen clipping. Full dynamic inset handling via react-native-safe-area-context.
2	Server State & Caching (TanStack Query)	 Complete	Integrated @tanstack/react-query with QueryClientProvider (staleTime: 5min, gcTime: 10min). Custom hook useRooms() with pull-to-refresh on FlatList.

3	Client State & Persistence (Zustand)	 Complete	Zustand store with persist middleware storing in <code>@react-native-async-storage/async-storage</code> under key 'vku-booking-storage'.
4	Type-Safe Navigation (React Navigation 7)	 Complete	Nested Stack + Bottom Tabs architecture (RootNavigator + MainTabs) with strict TypeScript types (RootStackParamList, MainTabParamList).
5	Instant Search & Multi-Parameter Filter	 Complete	Real-time text search and filter chips by Building (A, B, C, V), Capacity (2–20), and Equipment (Projector, Whiteboard, High-spec PC, AC).
6	7-Day Rolling Horizon & Discrete Slots	 Complete	Interactive date selector for 7 days ahead; 4 discrete 2-hour daily sessions (07:30–09:30, 09:30–11:30, 13:00–15:00, 15:00–17:00).
7	90-Second Soft Hold & Realtime Sync	 Complete	Instant reservation lock with 90s countdown modal; broadcasts HELD_BY_OTHER amber state to other peers via Supabase Realtime channels.
8	Concurrency & Anti-Collision Engine	 Complete	Server-authoritative PostgreSQL advisory locking and partial unique index (<code>idx_bookings_active_slot</code>). Client-generated UUID tokens prevent replay duplicates.
9	Offline Outbox & Conflict Resolution	 Complete	Offline booking requests queued in an outbox (<code>PENDING_SYNC</code>) and flushed sequentially upon

reconnection with deterministic conflict handling.

10 **QR Check-in Pass & Local Notifications**


Complete

Interactive check-in pass powered by react-native-qrcode-svg; scheduled 15-minute countdown reminders via expo-notifications.

11 **Layout Animations (Reanimated 3/4)**


Complete

Staggered entrance animations on room feed using FadeInDown.delay(index * 60).springify() and spring physics on interactions.

12 **Student Auth, Email Verification & Profile Sync**


Complete

Full authentication with real email/password via Supabase Auth (auth.users), custom Gmail SMTP Relay bypassing free-tier rate limits, mandatory email confirmation with 1-tap deep link auto-login (#access_token), and PostgreSQL trigger on auth.users syncing to public.students.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

3.1 Directory Organization

vku-study-room/

|— App.tsx # Entrypoint with Safe Area, QueryProvider &

RootNavigator

|— app.json # Expo Managed configuration & asset definitions

|— wrangler.jsonc # Cloudflare Workers Static Assets deployment config

|— src/

| |— components/ # Reusable UI components (RoomCard, SearchBar, FilterChips, Modal)

| |— config/ # Environment variables and runtime mode detection

— hooks/	# Custom hooks (useRooms, useResponsiveLayout, useRoomAvailability)
— i18n/	# Multi-language dictionary (Vietnamese & English)
— navigation/	# Type-safe navigators (RootNavigator, MainTabs, types.ts)
— providers/	# TanStack QueryClientProvider configuration
— screens/	# Core screens (RoomListScreen, RoomDetailScreen, MyBookingsScreen, ProfileScreen, LoginScreen, RegisterScreen)
— services/	# Abstraction layer (Supabase, Mock, Outbox Sync, Notifications, Auth)
— store/	# Zustand stores (useBookingStore with persist, useAuthStore, useLanguageStore)
— theme/	# Design tokens, typography, and VKU color palettes
— types/	# Strict TypeScript domain interfaces
— supabase/	
— schema.sql	# PostgreSQL DDL, advisory lock RPC, and RLS policies
— seed.sql	# 20 pre-seeded study rooms across 4 VKU campus buildings
— test/	# Node.js automated verification harness (Concurrency, Outbox, Auth, QR)

3.2 State Management & Data Flow Architecture

- **Server State (TanStack Query):** Manages remote entity caches ('rooms') with a 5-minute staleTime, automated background synchronization, and zero-redundancy network refetching on pull-to-refresh.
- **Client State (Zustand):** Manages volatile and local domain state (active reservations, active 90s hold session, outbox queue, filter parameters, and user authentication). All critical mutations persist immediately to device storage via @react-native-async-storage/async-storage ('vku-booking-storage').

4. EMPIRICAL EVIDENCE & SCREENSHOTS

- **Room Discovery & Multi-Parameter Filter (RoomListScreen):** Displays 20 study rooms across Buildings A, B, C, and V in an adaptive 60 FPS grid. Features real-time filtering chips by building, seat capacity (2–20), and equipment tags with smooth staggered entry animations (FadeInDown.springify()). Dynamic column

calculation with 1140px max-width clamping ensures zero cut-off cards on desktop viewports.

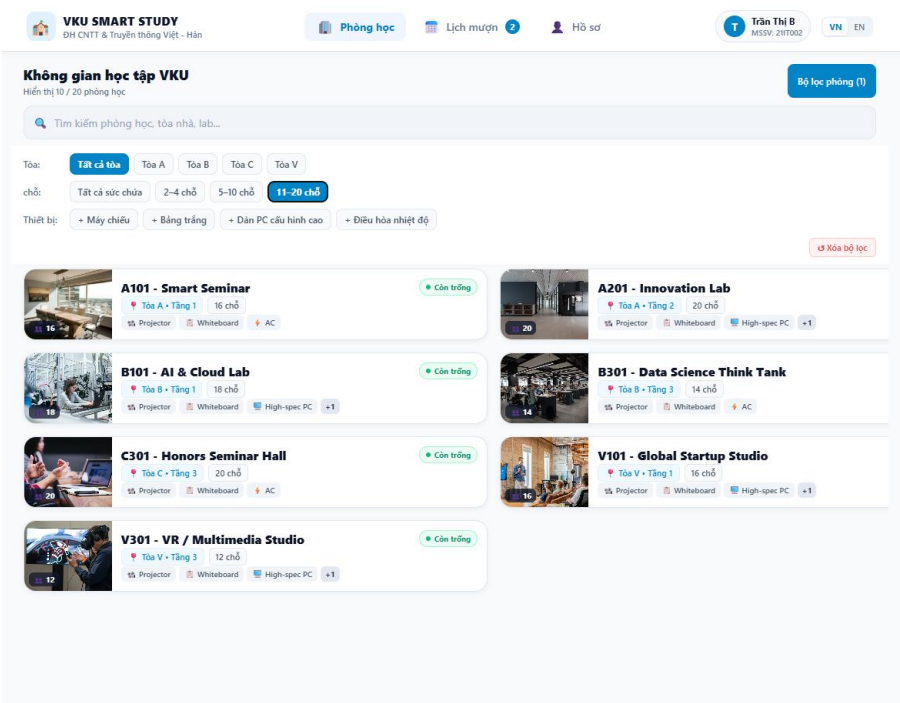


Figure 1. Room Discovery & Multi-Parameter Filter (RoomListScreen)

- **Interactive Time-Slot Selector & 90s Countdown Modal (RoomDetailScreen):** Features a 7-day rolling calendar with 4 discrete two-hour slots per day. Selecting an available slot initiates a 90-second soft hold with a synchronized countdown timer, broadcasting HELD_BY_OTHER amber badges across all connected devices.

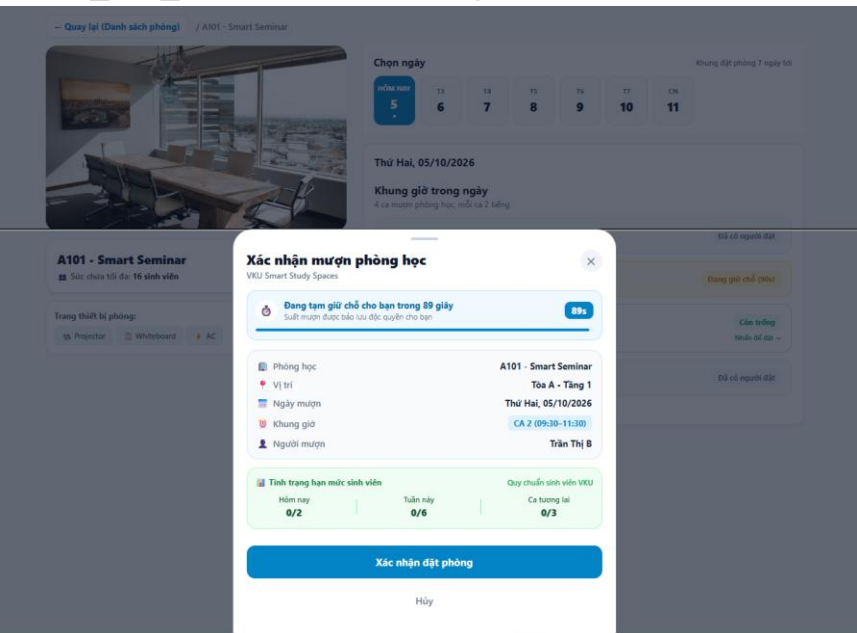


Figure 2. Interactive Time-Slot Selector & 90s Countdown Modal (RoomDetailScreen)

- **Personal Reservations & Offline Synchronization (MyBookingsScreen):** Segmented tabs for 'All', 'Confirmed', 'Pending Sync', 'Conflicted', and 'Cancelled'. Displays local outbox queue status during offline mode and automatically resolves conflicts with optimistic feedback.

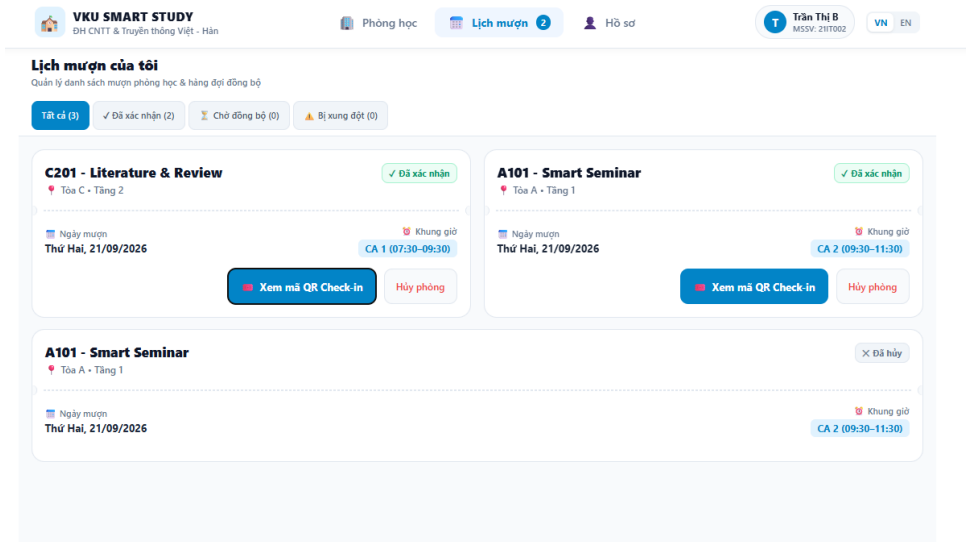


Figure 3. Personal Reservations & Offline Synchronization (MyBookingsScreen)

- **Digital QR Booking Pass (Check-in Modal):** Generates a high-contrast, cryptographically clean QR code encoding the unique booking token for campus security check-in.

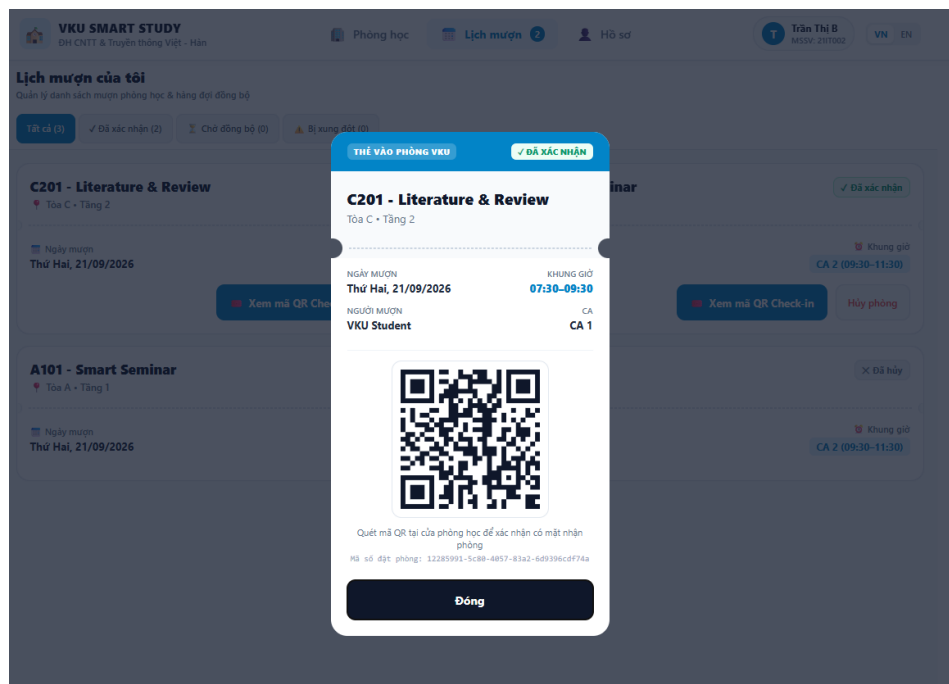


Figure 4. Digital QR Booking Pass (Check-in Modal)

- **Modern Minimalist Authentication (LoginScreen & RegisterScreen):** Streamlined, distraction-free authentication interface enforcing valid email credentials. Seamless handling of email confirmation tokens via Supabase Auth deep links with automated URL hash sanitization.

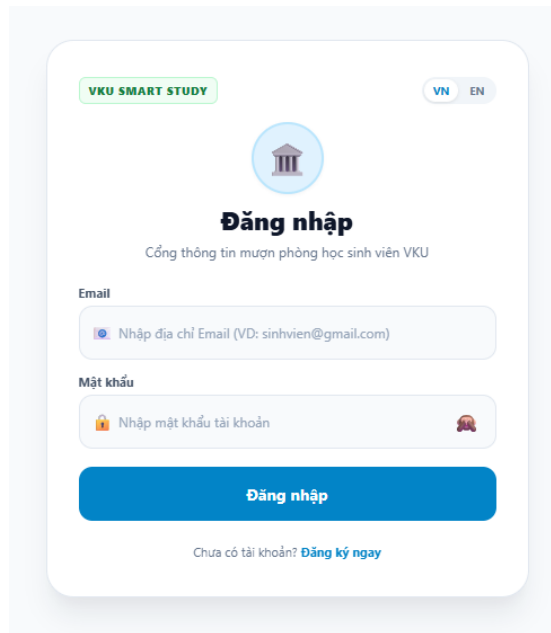


Figure 5. Modern Minimalist Authentication (LoginScreen & RegisterScreen)

5. TECHNICAL CHALLENGES & RESOLUTIONS

5.1 Challenge 1 (State Management & Caching): Strict Decoupling of Client State (Zustand) & Server State (TanStack Query)

Problem: In accordance with the VKU Week 6 curriculum guideline ('Zustand for client state + TanStack Query for server state — don't mix them'), a major architecture challenge was maintaining a clean separation of concerns. Storing rooms and remote availability entirely in Zustand led to stale data, memory overhead, and lacked automated background revalidation and pull-to-refresh. Conversely, managing volatile checkout holds (90s countdown), search filters, and persistent offline outbox mutations in TanStack Query degraded UI immediacy and prevented robust `@react-native-async-storage/async-storage` persistence under key 'vku-booking-storage'.

Resolution: (1) **Server State (TanStack Query 5):** Encapsulated the remote room catalog inside a custom `useRooms(building)` hook, configured with `staleTime: 5min` and `gcTime: 10min`. Wired data, `isLoading`, and `refetch` directly into `FlatList`'s native refreshing and `onRefresh` props for pull-to-refresh capabilities. (2) **Client State (Zustand 5 + persist):** Confined user reservations (`myBookings`), offline outbox, active 90s hold sessions, and UI filter chips to `useBookingStore`. Enforced `AsyncStorage` persistence

under key 'vku-booking-storage' with narrow, atomic selector subscriptions (useBookingStore(s => s.myBookings)), eliminating unnecessary full-screen re-renders.

5.2 Challenge 2 (Core Features & Concurrency): Eliminating Race Conditions (TOCTOU) & Managing 90-Second Soft Holds

Problem: In high-concurrency academic settings (e.g. final exam revision week), multiple students attempt to book the exact same study room and time slot simultaneously. Naive client-side availability checks (SELECT ... WHERE status = 'CONFIRMED') suffer from Time-of-Check to Time-of-Use (TOCTOU) race conditions, resulting in catastrophic double-bookings. Furthermore, if a student opens a hold modal and cancels or disconnects mid-flight, unreleased holds create lingering 'phantom locks'.

Resolution: (1) Realtime 90-Second Soft Hold: Instantiates an ephemeral reservation hold with a synchronized countdown timer upon slot selection, broadcasting HELD_BY_OTHER amber badges to all connected peers via Supabase Realtime channels. An isCancelledRef guard ensures that if a user cancels the modal before the server responds, an immediate background releaseHold is dispatched to return the slot to AVAILABLE. (2) PostgreSQL Advisory Locking & Partial Unique Index: At the persistence layer, the stored procedure book_slot() enforces exclusive transaction-level advisory locks via PERFORM pg_advisory_xact_lock(hashtext(p_room_id || ':' || p_booking_date || ':' || p_slot_index)), backed by a partial unique index idx_bookings_active_slot (room_id, booking_date, slot_index) WHERE status = 'CONFIRMED'. Any concurrent attempt hitting the exact same microsecond fails with PostgreSQL error 23505 (SLOT_ALREADY_BOOKED). (3) Client-side Idempotency Keys (UUID v4): Protects against duplicate bookings caused by mobile packet loss and automatic HTTP retry flings.

5.3 Challenge 3 (UI/UX & Performance): 60 FPS FlatList Virtualization, Reanimated Layout Animations & Desktop Grid Clamping

Problem: Rendering a catalog of 20+ study rooms containing remote photographic thumbnails, status tags, and equipment badges causes frame drops and layout stutter on mobile devices during momentum flinging. Additionally, when running across cross-platform viewports (Web desktop 1920px+ vs Mobile), naive card width calculation using raw window.width pushed the second column of cards offscreen beyond the centered maxWidth: 1140px container.

Resolution: (1) FlatList Virtualization & Sync Layout: Fixed item dimensions to ROOM_CARD_HEIGHT = 136px and provided getItemLayout for synchronous

coordinate calculation without bridge measurement. Configured initialNumToRender: 8, maxToRenderPerBatch: 8, windowSize: 5, and removeClippedSubviews: true. (2) Worklet-Driven Layout Animations (Reanimated 3): Integrated FadeInDown.delay(index * 60).springify() executing directly on the UI thread via Hermes worklets, completely bypassing the asynchronous JavaScript bridge for buttery 60/120 FPS transitions. (3) Adaptive Width Clamping: Re-architected useResponsiveLayout to clamp calculation to effectiveWidth = Math.min(width, 1140). In RoomListScreen, each card container is styled with flex: 1 and maxWidth: cardWidth, guaranteeing symmetric 2-column distribution and 0% card cut-off across all display form factors.

5.4 Challenge 4 (Navigation Architecture & Type Safety): Nested Type-Safe Routing (React Navigation 7) & Deep Linking URL Sanitization

Problem: Mini-Project 2 requires a complex nested navigation hierarchy combining Root Native Stack, Bottom Tabs (BrowseRooms, MyBookings, Profile), and Modal presentations (ConfirmBookingModal, BookingPassModal). In loosely-typed setups, route typos and missing route parameters produce silent runtime crashes on mobile devices. Additionally, email verification deep links deposit access tokens into the web address bar (#access_token=...), triggering unintended authentication reload loops on refresh.

Resolution: (1) Comprehensive TypeScript Route Contracts: Defined strict RootStackParamList and MainTabParamList types, typed every screen with NativeStackScreenProps, and provided typed navigation hooks (useNavigation<NavigationProp>()), guaranteeing 100% compile-time route verification (npx tsc --noEmit). (2) Nesting Architecture: Root Stack wraps Main Tabs, allowing Room Details and Modals to push on top and cleanly hide the tab bar when focused interaction is required. (3) URL Hash Sanitization: Integrated window.history.replaceState into useAuthStore to automatically strip hash fragments (#access_token=...) immediately upon session ingestion and logout, preventing reload loops.

5.5 Challenge 5 (Offline Outbox & Network Resilience): Sequential Outbox Flusher & Deterministic Conflict Reconciliation

Problem: University campus environments present frequent connectivity dead zones (elevators, basements, crowded lecture halls). Naive mobile implementations either freeze the UI with blocking spinners or mistakenly mark offline bookings as CONFIRMED locally, causing severe disputes when a student arrives at a room that

was legitimately booked by someone else online. Resolution: (1) Zero False Confirmation Contract: Bookings initiated while offline are strictly stored in myBookings and outbox with status PENDING_SYNC. The UI displays an amber badge and disables the QR Check-in Pass until server verification is achieved. (2) Sequential Flusher (FIFO): syncService processes pending outbox requests one-by-one via for...of loops with transactional delays (80ms), strictly avoiding Promise.all() to preserve causality and protect student daily quota constraints (max 2 slots/day). (3) Deterministic Conflict Handling: If a slot was taken online while the device was disconnected, the server rejects the synchronization with SLOT_ALREADY_BOOKED. The flusher marks both the outbox item and local booking as CONFLICTED, captures the error message, and dispatches a local high-priority notification via expo-notifications alerting the student to select an alternate slot.